

Contrato Operativo y de Protocolo — Orden de Servicio

CUY6142 — Telepresencia y Entornos Innovadores de Colaboración Humana

1. Información del Documento

CAMPO	VALOR
Documento	Contrato Operativo y de Protocolo — Orden de Servicio (API Demo eTOM/SOM)
Versión	1.3
Fecha	17/09/2026
Preparado por	Pablo Moscoso
Curso	CUY6142 — Telepresencia y Entornos Innovadores de Colaboración Humana
Actividad relacionada	EA2 — Fundamentos y Consumo de APIs (complementa 2.3/2.4)
Documento asociado	Contrato de Datos — Orden de Servicio

Gestión de Versiones

VERSIÓN	FECHA	PREPARADO POR	SECCIÓN Y TEXTO REVISADO
1.0	14/09/2026	Pablo Moscoso	Versión inicial
1.1	14/09/2026	Pablo Moscoso	Validación de conformidad contra <code>Estandar_Documentacion_API_CUY6142.md</code> (Sección 3, checklist de Ficha de Contrato); agregada Sección 10 "Control de Versiones del Contrato", ausente respecto a la Sección 1.5 del estándar — Glosario reenumerado de 10 a 11

VERSIÓN	FECHA	PREPARADO POR	SECCIÓN Y TEXTO REVISADO
1.2	14/09/2026	Pablo Moscoso	Agregado método <code>PUT /ordenes/{id}</code> (reemplazo completo de campos editables) a la Sección 5, con nota explicativa de la diferencia con <code>PATCH</code> ; actualizadas Secciones 7 (códigos de estado) y 8 (idempotencia) en consecuencia. No se agregó "UPDATE" — no es un método HTTP válido (ver respuesta al docente)
1.3	17/09/2026	Pablo Moscoso	Sección 4 rediseñada: el modelo de autenticación pasa de una clave única compartida a un mapa de dos claves por canal (<code>GUI</code> , <code>CRM</code>); la clave histórica se conserva sin cambios para el canal <code>GUI</code> , cambio aditivo documentado como menor (ver nota de excepción en Sección 11). Agregada Sección 6, "Webhooks — Notificaciones Asíncronas de Cambio de Estado" (registro, condiciones de disparo, modelo de entrega, payload, autenticación saliente, manejo de fallos y conciliación). Agregados endpoints <code>POST /webhooks</code> y <code>GET /webhooks/fallos</code> a la Sección 5. Renumeradas Secciones 6 a 11 → 7 a 12

2. Propósito y Alcance

Este documento define **cómo** un cliente interactúa con el servicio: transporte, autenticación, endpoints, formato de mensajes y códigos de estado. La estructura de los datos que viajan en cada mensaje (los campos de la Orden de Servicio) se define por separado en el **Contrato de Datos — Orden de Servicio**; este documento no repite esa definición.

El servicio simplifica el proceso eTOM **Order Handling** (Process Identifier 1.1.1.5), inspirado en el tipo de interacción cliente-servidor de un sistema real de Service Order Management (SOM), y alineado conceptualmente con el API abierto TM Forum **TMF641**, sin pretender conformidad ni certificación con ese estándar.

3. Modos de Despliegue y Host

El servicio opera en dos perfiles alternativos. Ambos usan el mismo puerto y el mismo contrato — solo cambia el host.

PERFIL	HOST	PUERTO	CUÁNDO SE USA
Demo en vivo	<IP_DOCENTE>	8081	Clase presencial. El docente comparte conexión a internet vía hotspot desde su teléfono; todos los dispositivos quedan en la misma red. El valor real de <IP_DOCENTE> se anuncia al inicio de la sesión — cambia en cada conexión al hotspot y no se documenta como valor fijo.
Distribuido	localhost (o 127.0.0.1)	8081	Cada estudiante despliega su propia instancia del servicio y prueba contra su propio equipo.

Base URL: `http://<HOST>:8081/api/v1`

Transporte: HTTP sin TLS. Es una limitación declarada, no una omisión: evita que cada estudiante deba generar y confiar un certificado autofirmado en el modo distribuido, lo que introduciría gestión de PKI ajena al objetivo del ejercicio. Consecuencia explícita: el header de autenticación (Sección 4) viaja en texto plano. Aceptable en el contexto de este ejercicio (red controlada en ambos perfiles), no aceptable como patrón para un despliegue en producción.

Fuera del alcance de este contrato: configuración de firewall del equipo servidor, límites de dispositivos conectados al punto de acceso, y aislamiento de clientes en la red utilizada. Son responsabilidad operativa de quien despliega el servicio (docente o estudiante) en el momento de uso, no una condición que el contrato de la API deba garantizar o documentar.

4. Autenticación

ASPECTO	DEFINICIÓN
Mecanismo	Header HTTP <code>X-API-Key</code>
Alcance de la clave	Una clave por canal de acceso — no una clave única para todo el curso, ni claves individuales por estudiante
Canales definidos	<code>GUI</code> (acceso manual vía Postman) y <code>CRM</code> (plataforma de gestión <code>crm-som</code>)
Valor documental	<code><API_KEY_CUY6142_GUI></code> y <code><API_KEY_CUY6142_CRM></code> — los valores reales los entrega el docente el día de uso
Continuidad con versiones anteriores	La clave usada históricamente en el canal <code>GUI</code> se conserva sin cambios; solo se agrega una clave nueva para el canal <code>CRM</code> . Ningún cliente existente deja de funcionar (ver Sección 11, Control de Versiones)

ASPECTO	DEFINICIÓN
Validez	Cada clave es válida en ambos perfiles de despliegue (demo en vivo y distribuido); no está atada al host, solo al canal
Momento de validación	Antes de procesar el cuerpo de la solicitud — es decir, antes de aplicar el Contrato de Datos
Falla	Header ausente o valor no presente en el mapa de claves válidas → 401 Unauthorized (Sección 8)

Distinción conceptual relevante para la clase: `X-API-Key` valida **quién llama** (nivel de protocolo/transporte); `cliente_id` en el cuerpo de la Orden de Servicio identifica **a nombre de quién se realiza la operación** (nivel de negocio, definido en el Contrato de Datos). Son dos conceptos independientes que en integraciones reales suelen confundirse.

Canal de origen. A partir de la versión 1.3, `X-API-Key` cumple un segundo rol además de autenticar: determina el **canal de origen** (`canal_origen`) de la solicitud, usado en el registro de log (Sección 8) y en el payload de los webhooks (Sección 6). El canal no es un valor que el cliente declara en el cuerpo de su solicitud — se deriva de cuál clave usó para autenticarse. Esta distinción es intencional: un campo declarado por el cliente puede escribirse con cualquier valor sin que el servidor pueda contradecirlo; un canal derivado de la clave con la que se autenticó solo puede falsificarse conociendo esa clave, lo que lo hace fiable como dato de auditoría y como criterio de negocio (por ejemplo, para distinguir en el payload de un webhook si la operación la originó el propio receptor u otro canal).

5. Endpoints y Métodos

MÉTODO	RUTA	ACCIÓN	ÉXITO	ERRORES POSIBLES
POST	<code>/ordenes</code>	Crear orden	201 Created	400, 401, 422
GET	<code>/ordenes</code>	Listar órdenes	200 OK	401
GET	<code>/ordenes/{id}</code>	Consultar una orden	200 OK	401, 404
PUT	<code>/ordenes/{id}</code>	Reemplazar completamente los campos editables de la orden	200 OK	400, 401, 404, 409, 422
PATCH	<code>/ordenes/{id}</code>	Transicionar estado	200 OK	401, 404, 409, 422

MÉTODO	RUTA	ACCIÓN	ÉXITO	ERRORES POSIBLES
DELETE	/ordenes/{id}	Cancelar orden	200 OK	401, 404, 409
POST	/webhooks	Registrar receptor de notificaciones asíncronas	201 Created	400, 401, 422
GET	/webhooks/fallos	Consultar intentos de notificación fallidos	200 OK	401

`PATCH` y `DELETE` exitosos disparan, además de su respuesta síncrona habitual, una notificación asíncrona (webhook) si hay un receptor registrado — ver Sección 6. Esa notificación no forma parte de la respuesta HTTP de `PATCH/DELETE`; es un efecto secundario documentado por separado, con su propio modelo de entrega y de fallos.

Diferencia entre `PUT` y `PATCH` en este contrato

Es la distinción central que se espera que el estudiante entienda de estos dos métodos:

- `PUT` reemplaza **todo** la representación editable de la orden (`cliente_id`, `tipo_servicio`, `prioridad`, `descripcion`). El cliente debe enviar los cuatro campos, incluso los que no cambian — un campo editable omitido se trata como ausente, igual que en una creación (Contrato de Datos, Sección 4). `PUT` es **idempotente**: enviar el mismo cuerpo dos veces deja la orden en el mismo estado final.
- `PATCH` en este contrato está deliberadamente acotado a una sola operación: transicionar `estado` según la máquina de estados del Contrato de Datos (Sección 5 de ese documento). No admite modificar `cliente_id`, `tipo_servicio`, `prioridad` ni `descripcion` — para eso se usa `PUT`.
- Ninguno de los dos métodos modifica `id`, `fecha_creacion` — son de solo lectura en todo momento. `PUT` tampoco modifica `estado`: si el cuerpo de un `PUT` incluye `estado`, el servidor lo ignora (Contrato de Datos, Sección 4).
- `PUT` solo se acepta si la orden está en `RECIBIDA` o `EN_PROGRESO`. Sobre una orden en `COMPLETADA` o `CANCELADA` responde `409 Conflict` — un estado terminal no admite edición de sus datos, la misma lógica que ya impide transicionar su `estado`.

Ejemplos de uso

Crear una orden:

```
curl -X POST http://<HOST>:8081/api/v1/ordenes \
  -H "X-API-Key: <API_KEY_CUY6142_CRM>" \
  -H "Content-Type: application/json" \
  -d '{"cliente_id":"CL-10457","tipo_servicio":"INTERNET","prioridad":"ALTA"}'
```

Consultar estado de una orden:

```
curl http://<HOST>:8081/api/v1/ordenes/<ID_ORDEN> \
  -H "X-API-Key: <API_KEY_CUY6142_CRM>"
```

Reemplazar completamente los campos editables de una orden (PUT):

```
curl -X PUT http://<HOST>:8081/api/v1/ordenes/<ID_ORDEN> \
  -H "X-API-Key: <API_KEY_CUY6142_CRM>" \
  -H "Content-Type: application/json" \
  -d '{"cliente_id":"CL-10457","tipo_servicio":"INTERNET","prioridad":"MEDIA","descripcion":"Cambio de prioridad tras contacto con el cliente"}'
```

Transicionar una orden a EN_PROGRESO:

```
curl -X PATCH http://<HOST>:8081/api/v1/ordenes/<ID_ORDEN> \
  -H "X-API-Key: <API_KEY_CUY6142_GUI>" \
  -H "Content-Type: application/json" \
  -d '{"estado":"EN_PROGRESO"}'
```

Estos mismos flujos son directamente reproducibles en Postman: header `X-API-Key`, cuerpo JSON en la pestaña Body → raw → JSON. Nótese que la clave usada en cada ejemplo corresponde al canal que ejecutaría esa operación en el diseño acordado: creación, consulta, `PUT` y `DELETE` desde el canal `CRM`; `PATCH` desde el canal `GUI` (Postman).

6. Webhooks — Notificaciones Asíncronas de Cambio de Estado

6.1 Propósito y alcance

Esta sección define el mecanismo por el cual el servicio informa a un receptor externo, por su propia iniciativa, que una orden cambió de estado — sin que el receptor deba consultar repetidamente (`polling`) para enterarse. Es el mismo patrón que usa Cisco Webex para notificar eventos (mensaje nuevo, sala creada) a una integración externa: el servidor llama al cliente, invirtiendo la dirección habitual de la comunicación.

No sustituye las respuestas síncronas ya definidas en la Sección 5 — es un mecanismo adicional. El servicio sigue respondiendo de inmediato, de forma síncrona, a cada solicitud (Sección 7, Formato de Mensajes); el webhook es un segundo aviso, posterior, sobre un cambio que ya ocurrió.

6.2 Registro del webhook

ASPECTO	DEFINICIÓN
Endpoint	<code>POST /api/v1/webhooks</code>
Autenticación	<code>X-API-Key</code> , igual que el resto de los endpoints (Sección 4)
Cuerpo de la solicitud	<code>{"url": "<url del receptor>"}</code>
Alcance	Un único webhook activo por servicio — registrar uno nuevo reemplaza al anterior; no se acumulan suscriptores
Persistencia	En memoria; se pierde al reiniciar el proceso, igual que las órdenes (Sección 10, Exclusiones Declaradas)
Validación	<code>url</code> es obligatorio y debe ser un string no vacío (422 si falta o está vacío). No se valida que la URL sea alcanzable en el momento del registro — solo se comprueba en el momento del disparo (Sección 6.4)

Registrar un receptor:

```
curl -X POST http://<HOST>:8081/api/v1/webhooks \
  -H "X-API-Key: <API_KEY_CUY6142_CRM>" \
  -H "Content-Type: application/json" \
  -d '{"url": "http://crm-som:8082/webhooks/ordenes"}'
```

6.3 Condiciones de disparo

El servicio dispara una notificación hacia el webhook registrado únicamente cuando:

- Una transición de estado vía `PATCH` se completa con éxito (`200`).
- Una cancelación vía `DELETE` se completa con éxito (`200`).

No se dispara en creación (`POST /ordenes`), consulta (`GET`) ni reemplazo (`PUT`) — estas operaciones no modifican el campo `estado`, que es la condición que activa la notificación. Un intento de `PATCH` o `DELETE` que falle (`401`, `404`, `409`, `422`) no dispara webhook: no hubo cambio de estado que notificar.

6.4 Modelo de entrega

La notificación se envía en un proceso separado del que atiende la solicitud original (`PATCH/DELETE`), con un timeout corto (3 segundos) y **sin reintentos automáticos**. El llamador original recibe su respuesta (`200`) sin esperar el resultado de la entrega del webhook — son dos operaciones desacopladas en el tiempo: la respuesta síncrona confirma que el estado cambió; la entrega del webhook, si ocurre, llega después y por un canal distinto.

Esta decisión es una limitación declarada, no una omisión, en el mismo espíritu que la ausencia de TLS (Sección 3) o de persistencia (Sección 10): un servicio de un solo proceso usado en clase no requiere la complejidad de una cola de reintentos con backoff exponencial, propia de un sistema de mensajería productivo. La Sección 6.7 documenta el mecanismo de conciliación que compensa la ausencia de reintentos.

Nota para quien implemente el receptor: si el receptor también expone endpoints propios que llaman de vuelta a este servicio, su servidor debe poder atender solicitudes entrantes de forma concurrente con sus propias solicitudes salientes — de lo contrario, un receptor de un solo hilo puede quedar interbloqueado esperando su propia respuesta mientras la notificación entrante espera turno detrás de ella. Es responsabilidad de implementación del receptor, fuera del alcance de este contrato.

6.5 Payload del evento

CAMPO	TIPO	DESCRIPCIÓN
<code>evento</code>	string	Tipo de evento. Valor fijo actual: <code>cambio_estado</code>
<code>orden_id</code>	string (UUID)	Identificador de la orden afectada
<code>estado_anterior</code>	enum	Estado antes de la operación
<code>estado_nuevo</code>	enum	Estado después de la operación

CAMPO	TIPO	DESCRIPCIÓN
operacion	enum (PATCH, DELETE)	Operación que disparó el evento
canal_origen	enum (CRM, GUI)	Canal que ejecutó la operación (Sección 4)
fecha_evento	string (ISO 8601)	Momento en que se disparó la notificación

Ejemplo de payload saliente:

```
{
  "evento": "cambio_estado",
  "orden_id": "5089a993-15f4-4237-b8b9-120141f41717",
  "estado_anterior": "EN_PROGRESO",
  "estado_nuevo": "COMPLETADA",
  "operacion": "PATCH",
  "canal_origen": "GUI",
  "fecha_evento": "2026-09-17T18:30:12Z"
}
```

El payload es deliberadamente liviano — no envía la representación completa de la orden (Contrato de Datos, Sección 3). El receptor que necesite más detalle puede hacer `GET /ordenes/{orden_id}` con el `orden_id` del evento. Es el mismo patrón que usan los webhooks de Webex: el evento trae el identificador del recurso, no su contenido completo.

6.6 Autenticación saliente

ASPECTO	DEFINICIÓN
Mecanismo	Header HTTP <code>X-Webhook-Secret</code> en la solicitud saliente hacia el receptor
Alcance	Único y fijo, compartido por todo el curso — mismo criterio de simplicidad ya aplicado a <code>X-API-Key</code> antes de la versión 1.3
Valor documental	<code><WEBHOOK_SECRET_CUY6142></code>
Propósito	El receptor puede validar que el callback entrante proviene efectivamente de este servicio, y no de un tercero que le llame directamente a su endpoint

Limitación declarada: con un secreto único y compartido, la validación prueba que el emisor conoce el secreto del curso, no que es específicamente este servicio frente a cualquier otro emisor que también lo conozca. Aceptable en este alcance — un solo emisor real, red controlada — igual que la limitación ya aceptada para `X-API-Key` en la Sección 4. No es una firma criptográfica (HMAC) del

cuerpo del mensaje, como usa Webex con su header `X-Spark-Signature` — es una comparación directa de secreto compartido. Una firma HMAC queda fuera de alcance de esta versión; es candidata natural para una extensión futura al llegar a la Actividad 3 del curso (Webex y sus APIs).

6.7 Manejo de fallos y conciliación

Si la entrega del webhook falla (timeout, conexión rechazada, o código de respuesta de error del receptor), el servicio registra el intento fallido tanto en el log del proceso (Sección 8) como en un registro consultable mediante:

```
GET /api/v1/webhooks/fallos
```

autenticado con `X-API-Key`, con esta forma de respuesta:

CAMPO	DESCRIPCIÓN
<code>orden_id</code>	Orden afectada por el cambio no notificado
<code>operacion</code>	<code>PATCH</code> o <code>DELETE</code>
<code>canal_origen</code>	Canal que ejecutó la operación original
<code>url_destino</code>	URL registrada en el momento del intento
<code>motivo</code>	Descripción del fallo (timeout, conexión rechazada, código de estado recibido)
<code>fecha_intento</code>	Momento del intento fallido

Este endpoint es el mecanismo de conciliación: dado que no hay reintentos automáticos (Sección 6.4), es responsabilidad del receptor, o de quien opera el servicio, consultarlo periódicamente y, ante una entrada, reconciliar el estado local contra `GET /ordenes/{id}`. Es el mismo patrón de "notificación push con red de seguridad por consulta" usado en integraciones productivas cuando el canal asíncrono no garantiza entrega.

Adicionalmente, toda respuesta de `PATCH` o `DELETE` que dispare un intento de notificación incluye el campo `notificacion_webhook: "pendiente"` en el cuerpo de la respuesta, cuando hay un webhook registrado. Es un aviso de que se intentará la notificación, no una confirmación de entrega: la naturaleza no bloqueante del envío (Sección 6.4) impide conocer el resultado antes de responder al llamador original.

Consultar entregas fallidas:

```
curl http://<HOST>:8081/api/v1/webhooks/fallos \
-H "X-API-Key: <API_KEY_CUY6142_CRM>"
```

6.8 Exclusiones declaradas de esta sección

- Reintentos automáticos de entrega (Sección 6.4).
- Firma criptográfica (HMAC) del payload saliente — se usa comparación directa de secreto compartido (Sección 6.6).
- Múltiples suscriptores simultáneos — solo un receptor activo a la vez (Sección 6.2).
- Deduplicación de eventos del lado del servidor — si el receptor la necesita, `orden_id` junto con `fecha_evento` permiten construir una clave de deduplicación propia; no es responsabilidad de este contrato.

7. Formato de Mensajes

- `Content-Type: application/json` obligatorio en toda solicitud con cuerpo (POST, PATCH).
- Codificación: UTF-8.
- Formato de error estandarizado, idéntico en todos los endpoints:

```
{
  "error": {
    "codigo": "TRANSICION_INVALIDA",
    "mensaje": "No se puede pasar de COMPLETADA a EN_PROGRESO"
  }
}
```

8. Códigos de Estado HTTP del Contrato

CÓDIGO	SIGNIFICADO EN ESTE CONTRATO	CUÁNDO SE PRODUCE
200 OK	Operación exitosa sobre un recurso existente	GET, PUT, PATCH, DELETE exitosos

CÓDIGO	SIGNIFICADO EN ESTE CONTRATO	CUÁNDO SE PRODUCE
201 Created	Recurso creado	POST exitoso (órdenes o webhooks)
400 Bad Request	El cuerpo de la solicitud no es JSON válido	Error de parseo, no de contenido
401 Unauthorized	Header <code>X-API-Key</code> ausente o no presente en el mapa de claves válidas	Falla de autenticación, antes de leer el cuerpo
404 Not Found	El <code>id</code> de orden no existe	GET, PUT, PATCH, DELETE sobre <code>id</code> inexistente
409 Conflict	Operación rechazada por el estado actual de la orden: transición de <code>estado</code> no permitida por la máquina de estados, o intento de PUT/DELETE sobre una orden en estado terminal (COMPLETADA/CANCELADA)	PUT, PATCH o DELETE que viola la Sección 5 del Contrato de Datos
422 Unprocessable Entity	JSON válido pero viola el Contrato de Datos (campo obligatorio ausente, valor fuera de enum), o el registro de webhook sin <code>url</code> válida	POST, PUT o PATCH de órdenes; POST de webhooks
500 Internal Server Error	Fallo no controlado del servidor	Excepción no anticipada

Distinción 400 vs 422: 400 significa que el cuerpo ni siquiera es JSON parseable; 422 significa que es JSON válido pero no cumple las reglas del Contrato de Datos (Sección 4 de ese documento) o del cuerpo esperado por el endpoint. Son dos capas de validación distintas y se comunican con códigos distintos.

9. Idempotencia y Concurrencia

- GET, PUT, DELETE y PATCH hacia un estado ya alcanzado son idempotentes: repetir la llamada con el mismo cuerpo dado dejan la orden en el mismo estado final. Matiz: `fecha_actualizacion` cambia en cada llamada exitosa aunque el resto de los campos no varíe — es un efecto secundario esperado, no invalida la idempotencia del recurso en sí.
- POST no es idempotente: cada llamada crea una nueva orden, incluso con el mismo cuerpo.
- El envío de notificaciones webhook tampoco es idempotente: cada PATCH o DELETE exitoso dispara un intento nuevo, independientemente de si un intento anterior tuvo éxito o falló.

No hay deduplicación de eventos definida por este contrato del lado del servidor (Sección 6.8) — si el receptor la necesita, es responsabilidad de su propia implementación.

- No hay control de concurrencia (sin bloqueo optimista ni versión de recurso). Es una limitación declarada, aceptable para un servicio de un solo proceso en memoria usado en clase, no para un entorno multiusuario real.
-

10. Exclusiones Declaradas del Contrato

- Persistencia real: almacenamiento en memoria, se reinicia al reiniciar el proceso — aplica tanto a las órdenes como al webhook registrado (Sección 6.2).
 - TLS/HTTPS (ver Sección 3).
 - Autenticación individual por estudiante (Sección 4) — el mapa de claves por canal introducido en la versión 1.3 sigue siendo un conjunto pequeño y fijo (GUI, CRM), no una clave por persona.
 - Configuración de red del entorno de ejecución: firewall, límites del punto de acceso, aislamiento de clientes (Sección 3) — responsabilidad operativa, no parte de este contrato.
 - Reintentos automáticos de entrega de webhook (Sección 6.4).
 - Firma criptográfica (HMAC) del payload saliente del webhook (Sección 6.6).
 - Múltiples suscriptores simultáneos de webhook (Sección 6.2).
-

11. Control de Versiones del Contrato

- Agregar un endpoint nuevo, o un código de estado adicional que no cambie el comportamiento de los existentes: cambio menor, no rompe clientes existentes.
- Cambiar el método HTTP de un endpoint existente, eliminar un endpoint, modificar el mecanismo de autenticación, o cambiar el significado de un código de estado ya definido: cambio mayor — rompe clientes existentes, requiere una nueva versión de la base URL (ej. /api/v2) y coordinación con el Contrato de Datos si el cambio también afecta la representación del recurso.

Aclaración aplicada en la versión 1.3: agregar una clave de autenticación nueva, sin modificar ni invalidar una clave ya existente, se documenta como cambio menor. El criterio general de esta sección clasifica "modificar el mecanismo de autenticación" como cambio mayor, pero esa regla apunta a *cambiar* o *eliminar* una forma de autenticación ya vigente — no a *agregar* una adicional que coexiste con la anterior sin afectarla. La clave históricamente usada por el canal GUI se mantiene sin cambios; ningún cliente existente deja de funcionar. Un cambio que sí ameritaría versión mayor bajo

este mismo criterio: eliminar la clave del canal `GUI`, cambiar su valor, o modificar el mecanismo de forma que invalide claves ya emitidas.

Esta ficha documenta la versión **1.3** del contrato operativo, correspondiente a la base URL `/api/v1`.

12. Glosario

TÉRMINO	DEFINICIÓN
Contrato Operativo	Documento que define cómo se accede a un servicio: transporte, autenticación, endpoints y manejo de errores.
Protocolo	Conjunto de reglas que rigen el intercambio de mensajes entre cliente y servidor (en este caso, HTTP + JSON).
Idempotencia	Propiedad de una operación cuyo resultado no cambia si se repite más de una vez con los mismos parámetros.
Header HTTP	Metadato enviado junto a una solicitud u respuesta HTTP, fuera del cuerpo del mensaje.
Status Code	Código numérico HTTP que indica el resultado de una solicitud.
Webhook	Mecanismo por el cual un servicio notifica, por su propia iniciativa, un evento a una URL registrada por el cliente, en vez de que el cliente deba consultar repetidamente (<code>polling</code>).
Callback	Sinónimo usado indistintamente con webhook en este documento: una llamada HTTP que el servidor realiza hacia el cliente, invirtiendo la dirección habitual de la comunicación.
Canal de origen	Identificador (<code>CRM</code> o <code>GUI</code>) que indica qué tipo de cliente ejecutó una operación, derivado de la clave de autenticación usada — no declarado por el cliente.
Conciliación	Proceso de comparar el estado local de un receptor contra el estado real del servicio, para detectar y corregir eventos que no llegaron a notificarse.

Documento de referencia — no es un procedimiento (MOP). Se actualiza cuando cambia el protocolo, la autenticación o los endpoints del servicio; los cambios en la estructura de datos se documentan en la ficha de Contrato de Datos asociada.